Τμήμα Πληροφορικής & Τηλεπικοινωνιών
Τεχνολογίες Εφαρμογών Διαδικτύου

NexusTickets

Εφαρμογή Διαχείρισης Εκδηλώσεων και
Ηλεκτρονικών Κρατήσεων στον Παγκόσμιο Ιστό

Μάριος Γιαννόπουλος
ΑΜ: 1115202000032

Ραμαντάν Κονόμι
ΑΜ: 1115201800281

Αθήνα, Ιούνιος 2026

Πίνακας Περιεχομένων

1 Εισαγωγή 1
2 Αρχιτεκτονική Συστήματος 1
3 Σχεδιασμός Βάσης Δεδομένων 1
4 Υλοποίηση Backend 1
4.1 Δομή Έργου 1
4.2 Διαμόρφωση (Configuration) 2
4.3 Βάση Δεδομένων 2
4.4 Επίπεδο Τομέα (Domain Layer) 3
4.5 Επίπεδο Αποθετηρίου (Repository Layer) 4
4.6 Επίπεδο Υπηρεσιών (Service Layer) 6
4.7 Επίπεδο Χειριστών (Handler Layer) 9
4.8 Ενδιάμεσο Λογισμικό (Middleware) 11
4.9 WebSocket 13
4.10 Εξαγωγή XML και JSON 14
4.11 Έγχυση Εξαρτήσεων (Dependency Injection) 14
5 Υλοποίηση Frontend 15
6 Αυθεντικοποίηση και Ασφάλεια 15
7 Σύστημα Συστάσεων 15
8 Δοκιμές 15
9 Επίλογος 15

1 Εισαγωγή

2 Αρχιτεκτονική Συστήματος

3 Σχεδιασμός Βάσης Δεδομένων

4 Υλοποίηση Backend

Το backend του NexusTickets υλοποιήθηκε εξ ολοκλήρου στη γλώσσα Go (έκδοση 1.22), αξιοποιώντας το πλαίσιο ανάπτυξης Fiber v2 για τη δημιουργία REST API και WebSocket διακομιστή. Η αρχιτεκτονική που ακολουθήθηκε είναι η Clean Architecture (Καθαρή Αρχιτεκτονική), η οποία διαχωρίζει την εφαρμογή σε τέσσερα διακριτά επίπεδα: το επίπεδο τομέα (domain layer), το επίπεδο αποθετηρίου (repository layer), το επίπεδο υπηρεσιών (service layer) και το επίπεδο χειριστών (handler layer). Ο διαχωρισμός αυτός εξασφαλίζει τη δυνατότητα δοκιμών (testability), τη συντηρησιμότητα του κώδικα και την ανεξαρτησία από εξωτερικά πλαίσια.

4.1 Δομή Έργου

Η δομή του backend ακολουθεί τις συμβάσεις της κοινότητας Go:

Κατάλογος	Περιγραφή	Ρόλος
`cmd/server/`	Είσοδος εφαρμογής	Εκκίνηση HTTP/Fiber server
`cmd/seed/`	Seeder δεδομένων	Εισαγωγή demo δεδομένων
`internal/config/`	Διαμόρφωση	Φόρτωση & επικύρωση env vars
`internal/domain/`	Οντότητες & Σταθερές	Μοντέλο πεδίου
`internal/dto/`	Data Transfer Objects	Αιτήσεις & Αποκρίσεις
`internal/repository/`	Αποθετήριο	Πρόσβαση σε βάση δεδομένων
`internal/service/`	Υπηρεσίες	Επιχειρηματική λογική
`internal/handler/`	Χειριστές	REST & WebSocket endpoints
`internal/middleware/`	Ενδιάμεσο λογισμικό	Auth, Rate limiting, CORS
`internal/redis/`	Πελάτης Redis	Σύνδεση & παραμετροποίηση
`internal/export/`	Εξαγωγή	XML & JSON streaming
`internal/wire/`	Dependency Injection	Wire providers & sets
`pkg/response/`	Απάντηση API	Envelope OK/Created/Error
`pkg/pagination/`	Σελιδοποίηση	Page/Meta helpers
`pkg/logger/`	Καταγραφή	Zap structured logger
`migrations/`	Μεταναστεύσεις	SQL scripts

Πίνακας 1: Δομή καταλόγων του backend

4.2 Διαμόρφωση (Configuration)

Η διαμόρφωση της εφαρμογής πραγματοποιείται μέσω μεταβλητών περιβάλλοντος (environment variables). Το πακέτο `internal/config/config.go` ορίζει τη δομή `Config` που περιέχει όλες τις ρυθμιζόμενες παραμέτρους της εφαρμογής, όπως τα στοιχεία σύνδεσης στη βάση δεδομένων, το secret για τα JWT tokens, τις ρυθμίσεις CORS, τις παραμέτρους του rate limiter και τη διαδρομή αποθήκευσης πολυμέσων.

Η φόρτωση γίνεται μέσω της συνάρτησης `Load()` η οποία:

- Καλεί την `loadDotEnv()` για προαιρετική φόρτωση αρχείου `.env` (μόνο σε περιβάλλον ανάπτυξης).
- Διαβάζει κάθε μεταβλητή περιβάλλοντος με τη συνάρτηση `getEnv(key, fallback)`.
- Αναλύει αριθμητικές παραμέτρους (ports, limits) με τις `parsePort`, `parseInt`, `parseDuration`.
- Επικυρώνει όλες τις τιμές χρησιμοποιώντας τη βιβλιοθήκη `go-playground/validator/v10`.
- Ελέγχει την ύπαρξη των αρχείων TLS (εφόσον έχουν οριστεί).

Παραδείγματα προεπιλεγμένων τιμών:

Μεταβλητή	Προεπιλογή	Περιγραφή
APP_ENV	development	Περιβάλλον εκτέλεσης
APP_PORT	8080	Θύρα HTTP server
DB_HOST	postgres	Διεύθυνση PostgreSQL
DB_MAX_OPEN_CONNS	25	Μέγιστες ταυτόχρονες συνδέσεις DB
RATE_LIMIT_MAX_REQUESTS	120	Μέγιστα requests ανά λεπτό
JWT_SECRET	dev-insecure-…	Secret για HMAC-SHA256

Πίνακας 2: Παράδειγμα μεταβλητών περιβάλλοντος και προεπιλογών

4.3 Βάση Δεδομένων

4.3.1 Σύνδεση

Το backend χρησιμοποιεί δύο αποθηκευτικά συστήματα:

1. **PostgreSQL 16** ως κύρια σχεσιακή βάση δεδομένων, με πρόσβαση μέσω GORM.
2. **Redis 7** για caching, rate limiting, αποθήκευση refresh token JTIs και μετρητές unread messages.

Η σύνδεση με την PostgreSQL γίνεται στο `internal/db/postgres.go` μέσω του GORM. Ορίζονται ρυθμίσεις για το pool συνδέσεων (`SetMaxOpenConns`, `SetMaxIdleConns`, `SetConnMaxLifetime`, `SetConnMaxIdleTime`) και πραγματοποιείται ping με timeout 3 δευτερολέπτων για επιβεβαίωση της σύνδεσης.

4.3.2 AutoMigrate

Η εξέλιξη της βάσης δεδομένων γίνεται μέσω του GORM AutoMigrate στο αρχείο `internal/db/automigrate.go`. Το AutoMigrate εκτελείται σε κάθε εκκίνηση του διακομιστή και δημιουργεί ή ενημερώνει τους πίνακες χωρίς απώλεια δεδομένων.

Μετά το AutoMigrate, εκτελούνται επιπλέον SQL εντολές με τη ρήτρα `IF NOT EXISTS` για:

- **10 CHECK constraints** — π.χ. `users_role_chk`, `events_capacity_chk`, `ticket_types_available_chk`, `bookings_quantity_chk`.
- **19 FOREIGN KEY constraints** — π.χ. `fk_events_organizer`, `fk_bookings_ticket_type`, `fk_messages_sender`, `fk_event_photos_event`.
- **3 UNIQUE composite indexes** — π.χ. `ticket_types_event_name_uq`, `event_categories_unique`.
- **1 generated column** — `search_vector tsvector GENERATED ALWAYS AS (...) STORED` με Greek full-text search configuration.
- **16+ performance indexes** — π.χ. `events_search_vector_gin_idx`, `events_startat_idx`, `messages_recipient_unread_idx`, `ticket_types_event_idx`.

Ο μηχανισμός `search_vector` χρησιμοποιεί `to_tsvector('greek', coalesce(title, '') || ' ' || coalesce(description, ''))` για την υποστήριξη αναζήτησης ελεύθερου κειμένου στα ελληνικά.

4.4 Επίπεδο Τομέα (Domain Layer)

Το επίπεδο τομέα βρίσκεται στο πακέτο `internal/domain/` και περιέχει τις βασικές οντότητες, τις σταθερές και τα σφάλματα της εφαρμογής.

4.4.1 Οντότητες

Οι κύριες οντότητες που ορίζονται είναι:

- **User** — Χρήστης με πεδία: ID (UUID v7), Username, PasswordHash, FirstName, LastName, Email, Phone, Address, City, Country, Latitude, Longitude, VATNumber, Role, AccountStatus, CreatedAt, UpdatedAt.
- **Event** — Εκδήλωση με πεδία: ID, OrganizerID, Title, Description, Venue, FeaturedImageURL, Detail (jsonb), Address, City, Country, Latitude, Longitude, StartAt, EndAt, Capacity, Status, CreatedAt, UpdatedAt.
- **TicketType** — Τύπος εισιτηρίου με πεδία: ID, EventID, Name, Price, Quantity, Available, CreatedAt, UpdatedAt.
- **Booking** — Κράτηση με πεδία: ID, UserID, EventID, TicketTypeID, Quantity, TotalCost, Status, CreatedAt, UpdatedAt.
- **Message** — Μήνυμα με πεδία: ID, SenderID, RecipientID, Subject, Body, ReadAt, DeletedBySender, DeletedByRecipient, CreatedAt.

- **Category** — Κατηγορία με πεδία: ID, Name (unique), CreatedAt.
- **EventPhoto** — Φωτογραφία εκδήλωσης με πεδία: ID, EventID, StoragePath, Position, CreatedAt.

Επιπλέον, ορίζεται η δομή `EventDetail` για πλούσιο περιεχόμενο εκδήλωσης (συντελεστές, καλλιτέχνες, χρήσιμες πληροφορίες, διοργανωτής, επαναλήψεις) και οι οντότητες για το chat: `ChatRoom`, `ChatRoomParticipant`, `ChatMessage`, `MessageReaction`, `Conversation`.

4.4.2 Σταθερές (Constants)

Οι σταθερές ορίζονται ως τύποι `string` και χρησιμοποιούν το pattern των const:

- **UserRole**: `admin`, `organizer`, `attendee`, `guest`
- **AccountStatus**: `pending`, `approved`, `rejected`
- **EventStatus**: `draft`, `published`, `cancelled`, `completed` (= `deleted`)
- **BookingStatus**: `pending`, `confirmed`, `cancelled`

4.4.3 Σφάλματα (Sentinel Errors)

Ορίζονται σφάλματα-φρουροί (sentinel errors) στο `domain/errors.go` που χρησιμοποιούνται σε όλα τα επίπεδα για ομοιόμορφο χειρισμό σφαλμάτων:

Σφάλμα	Περιγραφή
`ErrNotFound`	Το αντικείμενο δεν βρέθηκε
`ErrUnauthorized`	Μη εξουσιοδοτημένη πρόσβαση
`ErrForbidden`	Απαγορευμένη ενέργεια
`ErrUsernameTaken`	Το username είναι ήδη καταχωρημένο
`ErrCapacityExceeded`	Υπέρβαση διαθέσιμων εισιτηρίων
`ErrEventNotBookable`	Η εκδήλωση δεν δέχεται κρατήσεις
`ErrEventNotDeletable`	Η εκδήλωση δεν μπορεί να διαγραφεί
`ErrTicketTypeLocked`	Ο τύπος εισιτηρίου έχει κρατήσεις

Πίνακας 3: Κύρια sentinel errors του domain

4.5 Επίπεδο Αποθετηρίου (Repository Layer)

Το επίπεδο αποθετηρίου βρίσκεται στο `internal/repository/` και ακολουθεί το μοτίβο Dependency Inversion: ορίζονται διασυνδέσεις (interfaces) στο `interfaces.go` και η υλοποίηση γίνεται μέσω GORM στο `internal/repository/postgres/`.

4.5.1 Διασυνδέσεις

Οι κύριες διασυνδέσεις που ορίζονται:

| **Διασύνδεση** | **Κύριες Μέθοδοι** | |—|—| | UserRepository | GetByID, GetByUsername, GetByEmail, Create, UpdateProfile, Approve, Reject, List, ListByIDs | | EventRepository | Create, GetByID, ListOrganizer,

ListPublic, Search, Update, Delete, SetStatus, SetCategories, SetTicketTypes, GetPhotos, ListForExport | | TicketTypeRepository | Create, GetByID, ListByEvent, Update, Delete, DecrementAvailabilityAtomic | | BookingRepository | Create, CreateBookingWithDecrement, GetByID, ListByUser, ListByEvent, ListConfirmedByEvent, ListDistinctConfirmedUserIDsByEvent | | MessageRepository | Create, CreateBatch, ListInbox, ListSent, DeleteForUser, MarkAsRead, GetUnreadCount | | CategoryRepository | Create, GetByID, GetByName, ListAll | | ChatRoomRepository | CreateRoom, GetRoomByID, GetOrCreateDirectRoom, GetRoomByEvent, ListUserRooms, AddParticipant, ListUserRooms, CreateMessage, GetMessages |

4.5.2 Υλοποίηση με GORM

Τα GORM models ορίζονται στο `postgres/models.go` και οι αντιστοιχίσεις model-προς-domain γίνονται στο `postgres/mappers.go`. Κάθε model χρησιμοποιεί UUID v7 ως πρωτεύον κλειδί.

4.5.3 Ατομικές Λειτουργίες (Atomic Operations)

Η κρίσιμη λειτουργία δημιουργίας κράτησης υλοποιείται με ατομικό τρόπο στο `booking_tx.go`. Η μέθοδος `CreateBookingWithDecrement` εκτελείται εντός μιας συναλλαγής βάσης δεδομένων και ακολουθεί την εξής διαδικασία:

1. Εκτελεί `UPDATE ticket_types SET available = available - $1 WHERE id = $2 AND available >= $1`.
2. Ελέγχει τον αριθμό των γραμμών που επηρεάστηκαν (RowsAffected).
3. Αν `RowsAffected == 0`, ελέγχει αν υπάρχει το ticket type (επιστρέφει `ErrNotFound` αν δεν υπάρχει ή `ErrCapacityExceeded` αν δεν υπάρχουν διαθέσιμες θέσεις).
4. Δημιουργεί την εγγραφή κράτησης (booking) με την υπολογισμένη τιμή `totalCost = price * quantity`.
5. Αν αποτύχει οποιοδήποτε βήμα, γίνεται rollback της συναλλαγής.

Με αυτόν τον τρόπο αποφεύγονται τα race conditions και διασφαλίζεται ότι δύο ταυτόχρονες κρατήσεις δεν θα υπερβούν τη διαθέσιμη χωρητικότητα.

4.5.4 Αναζήτηση Πλήρους Κειμένου

Η αναζήτηση εκδηλώσεων υλοποιείται στο `EventRepository.Search` και υποστηρίζει:

- Αναζήτηση με `ILIKE` στον τίτλο.
- Αναζήτηση ελεύθερου κειμένου με `plainto_tsquery('greek', ...)` στο πεδίο `search_vector` (GIN index).
- Φιλτράρισμα κατά κατηγορία (μέσω του πίνακα `event_categories`).
- Φιλτράρισμα κατά χρονικό διάστημα (overlap check).
- Φιλτράρισμα κατά εύρος τιμής (μέσω EXISTS subquery σε `ticket_types`).

- Keyset pagination: το αποτέλεσμα περιλαμβάνει `after_start` και `after_id` για την επόμενη σελίδα.

4.6 Επίπεδο Υπηρεσιών (Service Layer)

Το επίπεδο υπηρεσιών βρίσκεται στο `internal/service/` και περιέχει όλη την επιχειρηματική λογική της εφαρμογής. Κάθε υπηρεσία δέχεται ως εξαρτήσεις τις διασυνδέσεις των αποθετηρίων και άλλες υπηρεσίες.

4.6.1 AuthService

Η υπηρεσία αυθεντικοποίησης διαχειρίζεται την εγγραφή, είσοδο, ανανέωση token και αποσύνδεση χρηστών.

Εγγραφή (`Register`):

1. Επικύρωση ταύτισης password και confirm_password.
2. Έλεγχος μοναδικότητας username και email (επιστροφή `ErrUsernameTaken` ή `ErrEmailTaken` αν υπάρχουν).
3. Κατακερματισμός password με bcrypt και κόστος 12.
4. Δημιουργία χρήστη με ρόλο `attendee` και κατάσταση `pending`.

Είσοδος (`Login`):

1. Αναζήτηση χρήστη βάσει username.
2. Σύγκριση bcrypt hash.
3. Έλεγχος κατάστασης λογαριασμού (πρέπει να είναι `approved`).
4. Έκδοση ζεύγους JWT tokens (access + refresh).

JWT Tokens:

- **Access token**: Διάρκεια 15 λεπτά, υπογεγραμμένο με HMAC-SHA256. Claims: `sub` (user ID), `role`, `token_type: "access"`, `jti` (τυχαίο UUID), `exp`, `iat`.
- **Refresh token**: Διάρκεια 7 ημέρες, HMAC-SHA256. Claims: `sub`, `role`, `token_type: "refresh"`, `jti`.
- Το JTI του refresh token αποθηκεύεται στο Redis με κλειδί `auth:refresh:<jti>` και TTL 7 ημερών.

Ανανέωση (`RefreshToken`):

1. Επαλήθευση JWT και έλεγχος `token_type == "refresh"`.
2. Επιβεβαίωση ύπαρξης JTI στο Redis.
3. Διαγραφή παλαιού JTI, έκδοση νέου ζεύγους token.

4.6.2 UserService

Διαχειρίζεται τα προφίλ χρηστών και τις ενέργειες διαχειριστή:

- `ListUsers` — Λίστα χρηστών με σελιδοποίηση.
- `GetUser` — Ανάκτηση στοιχείων χρήστη.

- `ApproveUser` / `RejectUser` — Έγκριση/απόρριψη εγγραφής από διαχειριστή.
- `UpdateProfile` — Μερική ενημέρωση προφίλ (όλα τα πεδία προαιρετικά).

4.6.3 EventService

Η πιο σύνθετη υπηρεσία, διαχειρίζεται ολόκληρο τον κύκλο ζωής των εκδηλώσεων.

Δημιουργία (`Create`):

1. Επικύρωση πεδίων (τίτλος, περιγραφή, ημερομηνίες, χωρητικότητα).
2. Έλεγχος: το άθροισμα των ποσοτήτων όλων των τύπων εισιτηρίων δεν πρέπει να υπερβαίνει τη συνολική χωρητικότητα της εκδήλωσης.
3. Δημιουργία εκδήλωσης με κατάσταση `draft`.
4. Δημιουργία τύπων εισιτηρίων.
5. Επίλυση ή δημιουργία κατηγοριών (μία εκδήλωση μπορεί να ανήκει σε πολλές κατηγορίες).

Δημοσίευση (`Publish`): Μεταβολή κατάστασης από `draft` σε `published`. Επιτρέπει στις κρατήσεις να ξεκινήσουν.

Ακύρωση (`Cancel`): Μεταβολή κατάστασης από `published` σε `cancelled`. Αποστέλλει αυτόματα ενημερωτικό μήνυμα σε όλους τους χρήστες με επιβεβαιωμένες κρατήσεις (μέσω `MessagingService.NotifyEventCancellation`). Νέες κρατήσεις αποκλείονται.

Διαγραφή (`Delete`): Επιτρέπεται μόνο αν η εκδήλωση βρίσκεται σε κατάσταση `draft`, ή αν είναι `published` αλλά δεν έχει καμία κράτηση.

Διαχείριση τύπων εισιτηρίων:

- `AddTicketType`, `UpdateTicketType`, `DeleteTicketType`.
- Έλεγχος χωρητικότητας σε κάθε δημιουργία/ενημέρωση.
- Αποκλεισμός τροποποίησης αν υπάρχουν κρατήσεις για τον συγκεκριμένο τύπο (`ErrTicketTypeLocked`).

Διαχείριση φωτογραφιών:

- Υποστήριξη upload (multipart), ανίχνευση MIME από bytes (jpeg, png, gif, webp), μέγιστο μέγεθος 5MB.
- Αποθήκευση στη διαδρομή `mediaRoot/<eventID>/<uuid>.<ext>`.
- Διαγραφή και ορισμός featured image.

4.6.4 BookingService

Υπηρεσία διαχείρισης κρατήσεων με τα εξής χαρακτηριστικά:

1. **Ατομικός έλεγχος διαθεσιμότητας** — Χρήση της ατομικής λειτουργίας `CreateBookingWithDecrement` για αποφυγή overbooking.

2. **Υπολογισμός κόστους από τον διακομιστή** — Το `totalCost` υπολογίζεται ως `price * quantity` από την πλευρά του server, χωρίς εμπιστοσύνη στην τιμή που στέλνει ο πελάτης.
3. **Έλεγχος κατάστασης εκδήλωσης** — Η κράτηση επιτρέπεται μόνο για εκδηλώσεις σε κατάσταση `published`.
4. **Πρόσβαση βάσει ρόλου** — Ο διοργανωτής βλέπει τις κρατήσεις των εκδηλώσεών του, ο συμμετέχων βλέπει μόνο τις δικές του.

4.6.5 MessagingService

Υπηρεσία ανταλλαγής μηνυμάτων μεταξύ διοργανωτή και συμμετέχοντα:

- `SendMessage` — Δημιουργία μηνύματος, αύξηση μετρητή unread στο Redis (`INCR nexustickets:unread:<userID>`), broadcast μέσω WebSocket Hub.
- `GetInbox` — Ανάγνωση εισερχομένων, αυτόματη σήμανση όλων ως αναγνωσμένων (`mark_all_read`), μηδενισμός μετρητή unread στο Redis.
- `GetSent` — Ανάγνωση απεσταλμένων χωρίς παρενέργειες.
- `GetUnreadCount` — Ανάκτηση μετρητή unread με cache-aside pattern: πρώτα Redis, αν miss τότε DB, αποθήκευση στο Redis.
- `DeleteForUser` — Λογική διαγραφή (setting flags `deleted_by_sender` / `deleted_by_recipient`).
- `NotifyEventCancellation` — Μαζική δημιουργία μηνυμάτων (batch INSERT) και broadcast μέσω WebSocket σε όλους τους χρήστες με επιβεβαιωμένες κρατήσεις.

4.6.6 RecommendationService

Η υπηρεσία συστάσεων ενσωματώνει τον αλγόριθμο Biased Matrix Factorization (BMF) για την πρόβλεψη ενδιαφέροντος χρήστη-εκδήλωσης. Η εκπαίδευση του μοντέλου γίνεται περιοδικά κάθε 6 ώρες μέσω goroutine ticker. Οι προβλέψεις αποθηκεύονται προσωρινά στο Redis με TTL 1 ώρας. Για χρήστες χωρίς ιστορικό παρέχεται fallback βάσει δημοφιλών εκδηλώσεων (cold-start). Η αντικατάσταση του μοντέλου γίνεται ατομικά με `sync.RWMutex`.

4.6.7 CatalogExportService

Υπηρεσία εξαγωγής δεδομένων σε μορφή XML και JSON, σύμφωνα με το DTD της εκφώνησης.

XML Export:

- Εγγραφή XML header και ρίζας `<eventCatalog>`.
- Ανάγνωση εκδηλώσεων σε batches των 100 εγγραφών.
- Για κάθε εκδήλωση: φόρτωση τύπων εισιτηρίων, κατηγοριών, κρατήσεων, χρηστών.
- Κωδικοποίηση με `xml.Encoder` με indentation.

- Ροή (streaming) του αποτελέσματος απευθείας στον πελάτη χωρίς προσωρινή αποθήκευση όλων των δεδομένων στη μνήμη.

JSON Export:

- Αντίστοιχη λειτουργία με streaming JSON batches και διαχωριστικά κόμματα.
- Μορφή: `{"events":[...]}`.

4.7 Επίπεδο Χειριστών (Handler Layer)

Οι χειριστές (handlers) βρίσκονται στο `internal/handler/` και είναι υπεύθυνοι για την επικοινωνία με τον πελάτη μέσω HTTP. Κάθε handler δέχεται DTOs (Data Transfer Objects) από το `internal/dto/` και ποτέ δεν εκθέτει οντότητες του domain layer απευθείας.

4.7.1 Ομαδοποίηση Διαδρομών (Routes)

Μέθοδος	Διαδρομή	Περιγραφή
POST	/api/v1/auth/register	Εγγραφή χρήστη
POST	/api/v1/auth/login	Είσοδος
POST	/api/v1/auth/refresh	Ανανέωση token
POST	/api/v1/auth/logout	Αποσύνδεση
GET	/api/v1/users/me	Στοιχεία χρήστη
PATCH	/api/v1/users/me	Ενημέρωση προφίλ
GET	/api/v1/users/admin	Λίστα χρηστών
PATCH	/api/v1/users/admin/:id/approve	Έγκριση χρήστη
PATCH	/api/v1/users/admin/:id/reject	Απόρριψη χρήστη
GET	/api/v1/events/	Λίστα εκδηλώσεων
GET	/api/v1/events/search	Αναζήτηση
GET	/api/v1/events/recommended	Προτεινόμενες
GET	/api/v1/events/:id	Λεπτομέρειες
POST	/api/v1/events/organizer	Δημιουργία εκδήλωσης
PATCH	/api/v1/events/organizer/:id	Ενημέρωση
DELETE	/api/v1/events/organizer/:id	Διαγραφή
POST	/api/v1/bookings/	Δημιουργία κράτησης
GET	/api/v1/bookings/me	Οι κρατήσεις μου
GET	/api/v1/messages/inbox	Εισερχόμενα
GET	/api/v1/messages/sent	Απεσταλμένα
POST	/api/v1/messages/	Αποστολή μηνύματος
GET	/api/v1/categories/	Λίστα κατηγοριών
GET	/api/v1/recommendations/	Συστάσεις χρήστη
GET	/api/v1/admin/events/export/xml	Εξαγωγή XML
GET	/api/v1/admin/events/export/json	Εξαγωγή JSON
GET	/api/v1/ws/messages	WebSocket για μηνύματα
GET	/api/v1/ws/chat	WebSocket για chat
GET	/health	Έλεγχος υγείας
GET	/ping	Ping

Πίνακας 4: Διαδρομές REST API

4.7.2 Data Transfer Objects (DTOs)

Όλοι οι χειριστές χρησιμοποιούν DTOs για την επικοινωνία με τον πελάτη:

Request DTOs:

- `RegisterRequest` — username, password, confirm_password, first_name, last_name, email, phone, address, city, country, vat_number.
- `LoginRequest` — username, password.
- `CreateEventRequest` — title, description, venue, address, city, country, latitude, longitude, start_at, end_at, capacity, category_ids, ticket_types.
- `CreateBookingRequest` — event_id, ticket_type_id, quantity.
- `SendMessageRequest` — recipient_id, subject, body.

Response DTOs:

- `AuthTokensResponse` — access_token, refresh_token, expires_in, user.
- `UserResponse` — Όλα τα πεδία χρήστη εκτός από password hash.
- `EventResponse` — Πλήρη στοιχεία εκδήλωσης με ticket types, categories, photos.
- `BookingResponse` — Στοιχεία κράτησης με event, ticket type, cost.
- `MessageResponse` — Στοιχεία μηνύματος με sender/recipient πληροφορίες.

4.7.3 Μορφή API Envelope

Κάθε απόκριση ακολουθεί την ομοιόμορφη μορφή:

```
{
  "data": { ... },
  "error": null,
  "meta": { "page": 1, "limit": 20, "total_items": 50, "total_pages": 3 }
}
```

4.8 Ενδιάμεσο Λογισμικό (Middleware)

Το ενδιάμεσο λογισμικό υλοποιείται στο `internal/middleware/` και εφαρμόζεται σε επίπεδο διαδρομής ή ομάδας διαδρομών.

4.8.1 JWTAuth

Το middleware `JWTAuth` επαληθεύει την παρουσία και εγκυρότητα του JWT token:

1. Διαβάζει την κεφαλίδα `Authorization: Bearer <token>`.
2. Αναλύει το JWT, επαληθεύει την υπογραφή HMAC-SHA256.
3. Ελέγχει ότι το `token_type == "access"`.
4. Εξάγει τα claims `sub` (user ID) και `role`.
5. Αποθηκεύει τα στοιχεία στο `c.Locals("user_id")` και `c.Locals("role")`.
6. Σε περίπτωση αποτυχίας, επιστρέφει HTTP 401.

Υπάρχει και η παραλλαγή `OptionalJWTAuth` που αποτυγχάνει σιωπηλά (δεν επιστρέφει σφάλμα αν δεν υπάρχει token), χρησιμοποιείται για διαδρομές όπως `GET /events/recommended` και `GET /events/:id` όπου η αυθεντικοποίηση είναι προαιρετική.

4.8.2 WebSocketJWTAuth

Παραλλαγή του JWTAuth για WebSocket συνδέσεις. Αντί για κεφαλίδα Authorization, διαβάζει το token από το query parameter `?token=<JWT>`.

4.8.3 RequireRole

Το `RequireRole(roles ...domain.UserRole)` είναι ένα guard που ελέγχει αν ο ρόλος του χρήστη (από `c.Locals("role")`) βρίσκεται στο επιτρεπόμενο σύνολο. Αν όχι, επιστρέφει HTTP 403.

4.8.4 RateLimiter

Υλοποίηση sliding window log με χρήση Redis Sorted Sets και Lua script:

1. `ZREMRANGEBYSCORE` — Αφαίρεση καταχωρήσεων εκτός χρονικού παραθύρου.
2. `ZADD` — Προσθήκη τρέχουσας χρονικής στιγμής.
3. `ZCARD` — Μέτρηση καταχωρήσεων εντός παραθύρου.
4. Αν ο αριθμός υπερβαίνει το όριο, επιστροφή HTTP 429.

Το κλειδί στο Redis είναι `rate_limit:<segment>:<IP>`. Διαφορετικά τμήματα (segments) χρησιμοποιούνται για διαφορετικές ομάδες διαδρομών (π.χ. `"auth"` με 5 requests/min, `"global"` με 120 requests/min).

4.8.5 CORS

Επιτρέπει συγκεκριμένα origins που ορίζονται στη μεταβλητή `CORS_ALLOWED_ORIGINS`. Οι επιτρεπόμενες μέθοδοι είναι GET, POST, PUT, PATCH, DELETE, OPTIONS. Υποστηρίζονται credentials και η κεφαλίδα Authorization.

4.8.6 ErrorHandler

Κεντρικός χειριστής σφαλμάτων που αντιστοιχίζει τα sentinel errors του domain σε κατάλληλους HTTP κωδικούς:

Sentinel Error	**HTTP Code**
`ErrUnauthorized`	401
`ErrForbidden`	403
`ErrNotFound`	404
`ErrUsernameTaken`, `ErrEmailTaken`, `ErrCapacityExceeded`	409
`ErrEventNotBookable`, `ErrInvalidEventState`, `ErrTicketTypeLocked`	422
Άγνωστο σφάλμα	500

Πίνακας 5: Αντιστοίχιση sentinel errors σε HTTP κωδικούς

4.9 WebSocket

Η επικοινωνία πραγματικού χρόνου (Real-time Communication) υλοποιείται μέσω του πρωτοκόλλου WebSocket, χρησιμοποιώντας τη βιβλιοθήκη Gorilla WebSocket σε συνδυασμό με Fiber. Η αρχιτεκτονική ακολουθεί το μοτίβο Hub/Client.

4.9.1 Hub

Το `Hub` είναι μια κεντρική δομή που διατηρεί έναν χάρτη συνδέσεων: `map[uuid.UUID]map[*Client]struct{}`. Κάθε χρήστης μπορεί να έχει πολλές ταυτόχρονες συνδέσεις (π.χ. από διαφορετικές καρτέλες του browser).

Η κύρια λειτουργία του Hub είναι ένα event loop που εκτελείται σε ξεχωριστό goroutine και διαχειρίζεται τρία κανάλια:

- `register` — Προσθήκη νέου client στη δομή.
- `unregister` — Αφαίρεση client και κλείσιμο του καναλιού `send`.
- `SendToUser(userID, payload)` — Αποστολή μηνύματος σε όλες τις ενεργές συνδέσεις του χρήστη (non-blocking send).

4.9.2 Client

Κάθε `Client` αντιπροσωπεύει μία WebSocket σύνδεση και διαθέτει:

- `ReadPump` — Διαβάζει μηνύματα από τη σύνδεση (η εφαρμογή είναι receive-only για τα μηνύματα inbox, τα οποία αποστέλλονται μέσω REST). Διαχειρίζεται pong responses και timeout.
- `WritePump` — Τικερ ping κάθε 54 δευτερόλεπτα (ping period = 9/10 του pong wait). Γράφει μηνύματα από το κανάλι `send` στη σύνδεση.
- Timeouts: writeWait = 10s, pongWait = 60s, pingPeriod = 54s.
- Μέγιστο μέγεθος μηνύματος: 4096 bytes.

4.9.3 Ροή Επικοινωνίας

Η ροή επικοινωνίας για τα μηνύματα έχει ως εξής:

1. Ο χρήστης συνδέεται στο `/api/v1/ws/messages?token=<JWT>` (ή `/api/v1/ws/chat`).
2. Το `WebSocketJWTAuth` επαληθεύει το JWT token.
3. Το `Hub.Register` αποθηκεύει τον νέο client.
4. Όταν ένας άλλος χρήστης στέλνει μήνυμα μέσω REST (`POST /api/v1/messages/`), η `MessagingService` καλεί `hub.SendToUser(recipientID, payload)`.
5. Το Hub διανέμει το μήνυμα σε όλες τις ενεργές συνδέσεις του παραλήπτη.
6. Ο `WritePump` του client γράφει το μήνυμα στη WebSocket σύνδεση.

Τα δύο είδη μηνυμάτων WebSocket που ορίζονται:

- `new_message` — Νέο μήνυμα στα εισερχόμενα (περιλαμβάνει όλα τα στοιχεία του μηνύματος).

- `unread_count` — Ενημέρωση μετρητή μη αναγνωσμένων μηνυμάτων.

4.10 Εξαγωγή XML και JSON

Η εξαγωγή δεδομένων σε μορφή XML και JSON υλοποιείται στο `internal/export/catalog.go` και ικανοποιεί πλήρως το DTD της εκφώνησης.

4.10.1 XML Export

Η δομή `EventExportXML` αντιστοιχεί ακριβώς στο DTD:

- Κάθε στοιχείο XML αντιστοιχεί σε πεδίο της δομής με XML tags.
- Οι ιδιότητες (attributes) όπως `EventID`, `TicketTypeID`, `BookingID` ορίζονται με `xml:"...,attr"`.
- Οι εμφωλευμένες δομές (TicketTypes, Bookings, Organizer, GeoLocation, Media, Attendee) ορίζονται ως ξεχωριστές δομές.
- Η εξαγωγή γίνεται σε ροή (streaming): ο server διαβάζει τις εκδηλώσεις σε batches (100 εγγραφές ανά batch) και γράφει απευθείας στον HTTP response writer με `SetBodyStreamWriter`.
- Το `Content-Disposition: attachment; filename=events.xml` ενημερώνει τον browser να κατεβάσει το αρχείο.

4.10.2 JSON Export

Αντίστοιχη δομή `EventJSON` με snake_case JSON tags, ίδια λογική streaming με batches.

4.11 Έγχυση Εξαρτήσεων (Dependency Injection)

Η έγχυση εξαρτήσεων γίνεται μέσω του εργαλείου `google/wire`. Στο `internal/wire/wire.go` ορίζονται:

- **RepositorySet** — Όλες οι υλοποιήσεις αποθετηρίων.
- **ServiceSet** — Όλες οι υπηρεσίες, με χρήση provider συναρτήσεων για όσες χρειάζονται ειδική παραμετροποίηση (π.χ. `provideAuthService` που δέχεται `cfg.JWTSecret`).
- **HandlerSet** — Όλοι οι χειριστές.

Η κεντρική συνάρτηση `InitializeApp(db, redis, cfg)` καλεί το `wire.Build` με όλα τα sets και το `wire.Struct(new(AppDependencies), "*")` για να παράγει αυτόματα τη δομή `AppDependencies` με όλες τις εξαρτήσεις.

Το παραγόμενο αρχείο `wire_gen.go` περιέχει την υλοποίηση της συνάρτησης χωρίς αντανακλάσεις (reflection), με στατικό τύπο και βελτιστοποιημένη απόδοση.

5 Υλοποίηση Frontend

6 Αυθεντικοποίηση και Ασφάλεια

7 Σύστημα Συστάσεων

8 Δοκιμές

9 Επίλογος